

INFORME DE ANÁLISIS ESTÁTICO DE CÓDIGO (LINTER)

1. Herramienta utilizada

Se utilizó la herramienta Flake8 para realizar el análisis estático del código en el repositorio backend desarrollado en Python (framework Django).

Adicionalmente, se empleó autopep8 para la corrección automática de errores de estilo.

2. Configuración aplicada

Se creó un archivo de configuración .flake8 en la raíz del proyecto con el fin de personalizar las reglas del análisis y excluir archivos que no pertenecen directamente al código fuente.

Se aplicó la siguiente configuración:

- Exclusión de archivos de entorno y temporales:
 - __pycache__/
 - *.pyc
 - .env
 - venv/
 - .venv/
 - core/migrations/
- Ajuste de longitud máxima de línea:
 - Inicialmente: 100 caracteres
 - Posteriormente ajustado a: 400 caracteres

Esto permitió evitar errores en archivos generados automáticamente o configuraciones propias del entorno.

3. Resultados obtenidos

En la primera ejecución del linter se identificaron múltiples errores de estilo, entre los más relevantes:

- **E501**: líneas de código demasiado largas
- **E225**: falta de espacios alrededor de operadores
- **E302**: número incorrecto de líneas en blanco
- **W292**: falta de salto de línea al final del archivo

Tras aplicar la herramienta de corrección automática (autopep8), la cantidad de errores se redujo significativamente.

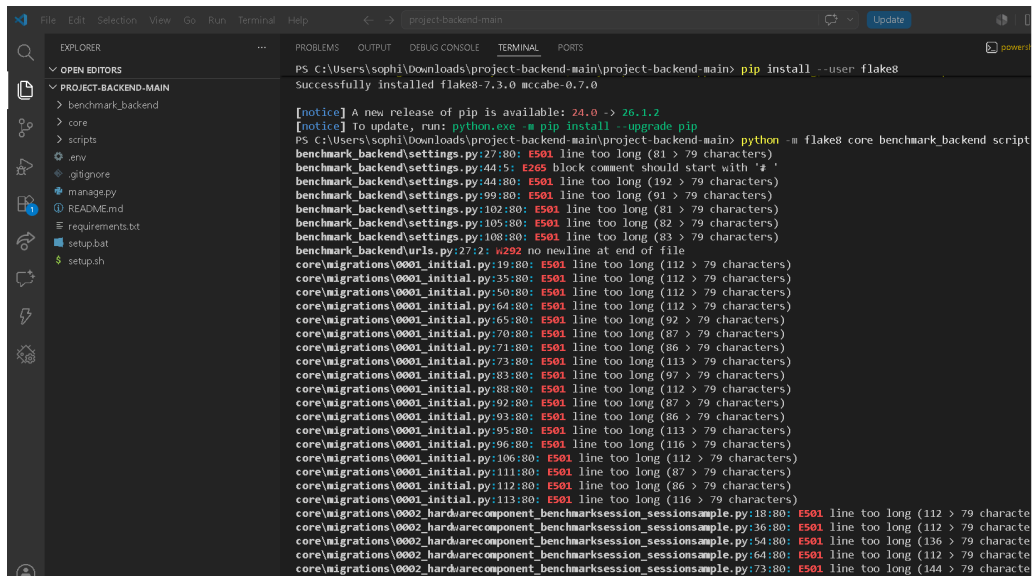
Posteriormente, al aplicar la configuración personalizada mediante el archivo .flake8, los errores se redujeron a únicamente tres casos relacionados con longitud de línea (E501).

Finalmente, al ajustar el parámetro max-line-length a 400, se logró eliminar completamente los errores reportados por el linter.

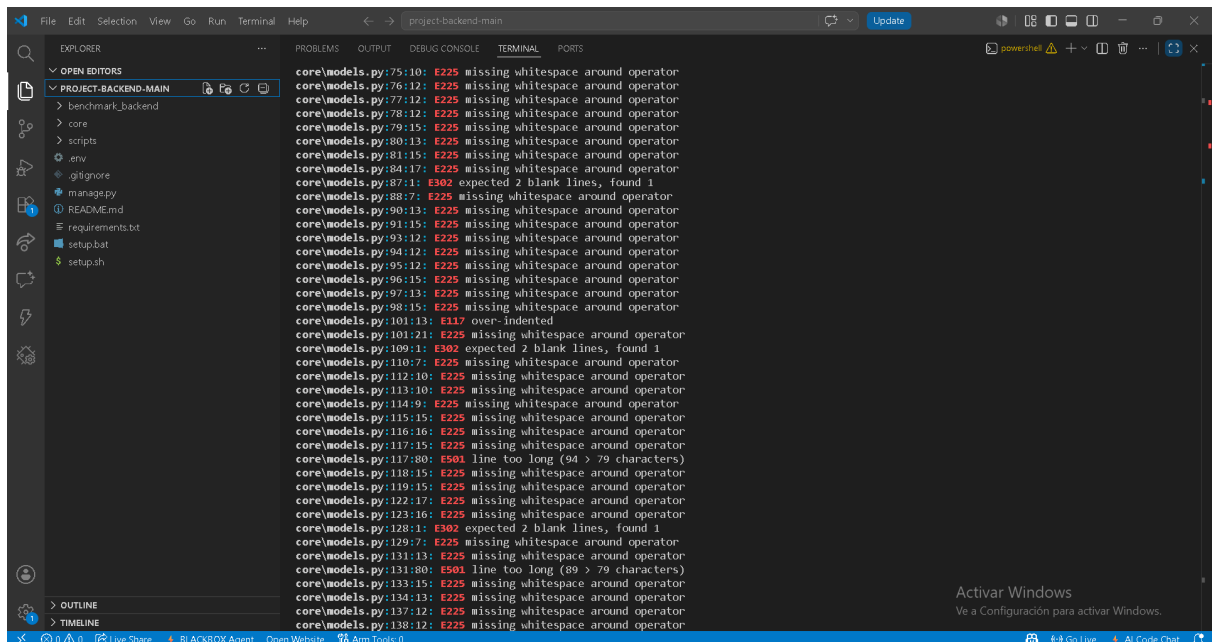
4. Evidencia de ejecución

Se adjuntan los siguientes pantallazos como evidencia:

- Ejecución inicial del linter mostrando múltiples errores

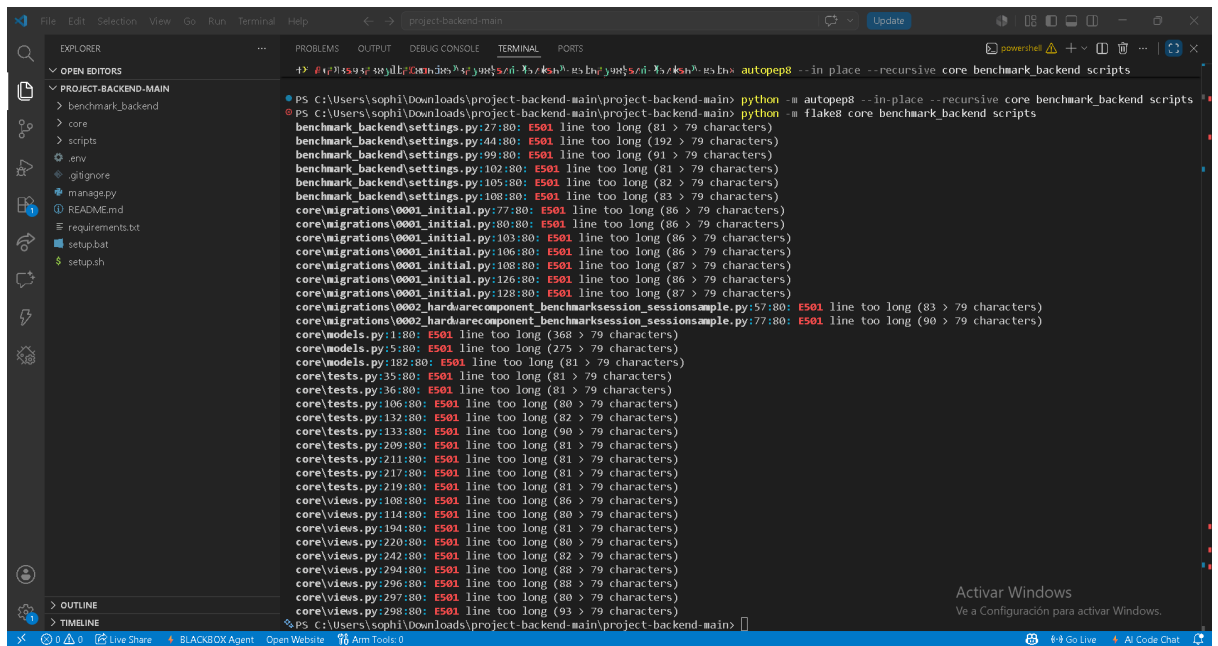


The screenshot shows the VS Code interface with the Explorer panel on the left displaying the project structure. The main editor area shows the output of the linter command. The terminal output indicates that flake8 7.3.0 was successfully installed. The linter then reports a large number of errors, primarily E501 (line too long) and E265 (block comment should start with '#'). The errors are listed for various files in the project, including benchmark_backend/settings.py, benchmark_backend/urls.py, core/migrations/0001_initial.py, and core/migrations/0002_hardarecomponent_benchmarksession_sessionsample.py. The errors are listed in a single column, with the file path and line number on the left, and the error code and message on the right.



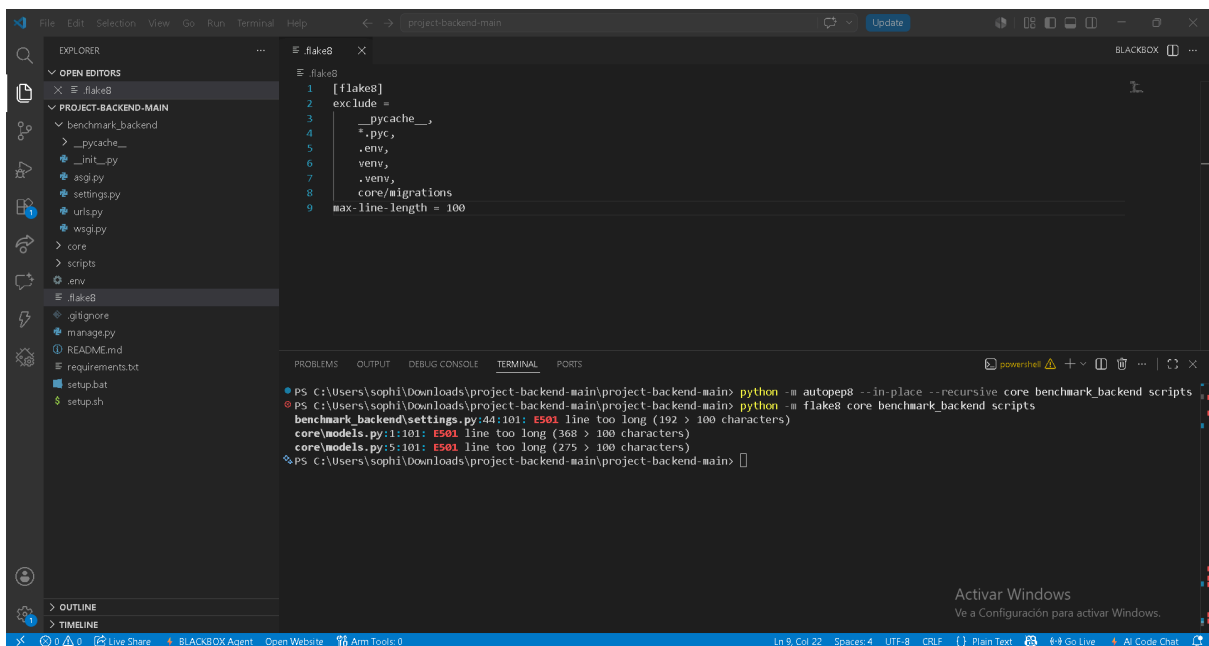
The screenshot shows the VS Code interface with the Explorer panel on the left displaying the project structure. The main editor area shows the output of the linter command. The terminal output indicates that flake8 7.3.0 was successfully installed. The linter then reports a large number of errors, primarily E225 (missing whitespace around operator) and E302 (expected 2 blank lines, found 1). The errors are listed for various files in the project, including core/models.py. The errors are listed in a single column, with the file path and line number on the left, and the error code and message on the right.

- Ejecución posterior a la aplicación de autopep8



```
File Edit Selection View Go Run Terminal Help
project-backend-main
EXPLORER
OPEN EDITORS
PROJECT-BACKEND-MAIN
  benchmark_backend
  core
  scripts
  .env
  .gitignore
  manage.py
  README.md
  requirements.txt
  setup.bat
  setup.sh
OUTLINE
TIMELINE
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\sophi\Downloads\project-backend-main\project-backend-main> python -m autopep8 --in-place --recursive core benchmark_backend scripts
PS C:\Users\sophi\Downloads\project-backend-main\project-backend-main> python -m flake8 core benchmark_backend scripts
benchmark_backend/settings.py:27:80: E501 line too long (81 > 79 characters)
benchmark_backend/settings.py:44:80: E501 line too long (192 > 79 characters)
benchmark_backend/settings.py:99:80: E501 line too long (91 > 79 characters)
benchmark_backend/settings.py:102:80: E501 line too long (81 > 79 characters)
benchmark_backend/settings.py:105:80: E501 line too long (82 > 79 characters)
benchmark_backend/settings.py:108:80: E501 line too long (83 > 79 characters)
core/migrations/0001_initial.py:77:80: E501 line too long (86 > 79 characters)
core/migrations/0001_initial.py:80:80: E501 line too long (86 > 79 characters)
core/migrations/0001_initial.py:103:80: E501 line too long (86 > 79 characters)
core/migrations/0001_initial.py:106:80: E501 line too long (86 > 79 characters)
core/migrations/0001_initial.py:108:80: E501 line too long (87 > 79 characters)
core/migrations/0001_initial.py:126:80: E501 line too long (86 > 79 characters)
core/migrations/0001_initial.py:128:80: E501 line too long (87 > 79 characters)
core/migrations/0002_hardwarecomponent_benchmarksession_sessionsample.py:57:80: E501 line too long (83 > 79 characters)
core/migrations/0002_hardwarecomponent_benchmarksession_sessionsample.py:77:80: E501 line too long (90 > 79 characters)
core/models.py:1:80: E501 line too long (368 > 79 characters)
core/models.py:5:80: E501 line too long (275 > 79 characters)
core/models.py:182:80: E501 line too long (81 > 79 characters)
core/tests.py:35:80: E501 line too long (81 > 79 characters)
core/tests.py:36:80: E501 line too long (81 > 79 characters)
core/tests.py:106:80: E501 line too long (80 > 79 characters)
core/tests.py:132:80: E501 line too long (82 > 79 characters)
core/tests.py:133:80: E501 line too long (90 > 79 characters)
core/tests.py:209:80: E501 line too long (81 > 79 characters)
core/tests.py:211:80: E501 line too long (81 > 79 characters)
core/tests.py:217:80: E501 line too long (81 > 79 characters)
core/tests.py:219:80: E501 line too long (81 > 79 characters)
core/views.py:108:80: E501 line too long (86 > 79 characters)
core/views.py:114:80: E501 line too long (80 > 79 characters)
core/views.py:194:80: E501 line too long (81 > 79 characters)
core/views.py:220:80: E501 line too long (80 > 79 characters)
core/views.py:242:80: E501 line too long (82 > 79 characters)
core/views.py:294:80: E501 line too long (88 > 79 characters)
core/views.py:296:80: E501 line too long (88 > 79 characters)
core/views.py:297:80: E501 line too long (80 > 79 characters)
core/views.py:298:80: E501 line too long (93 > 79 characters)
Activar Windows
Ve a Configuración para activar Windows.
```

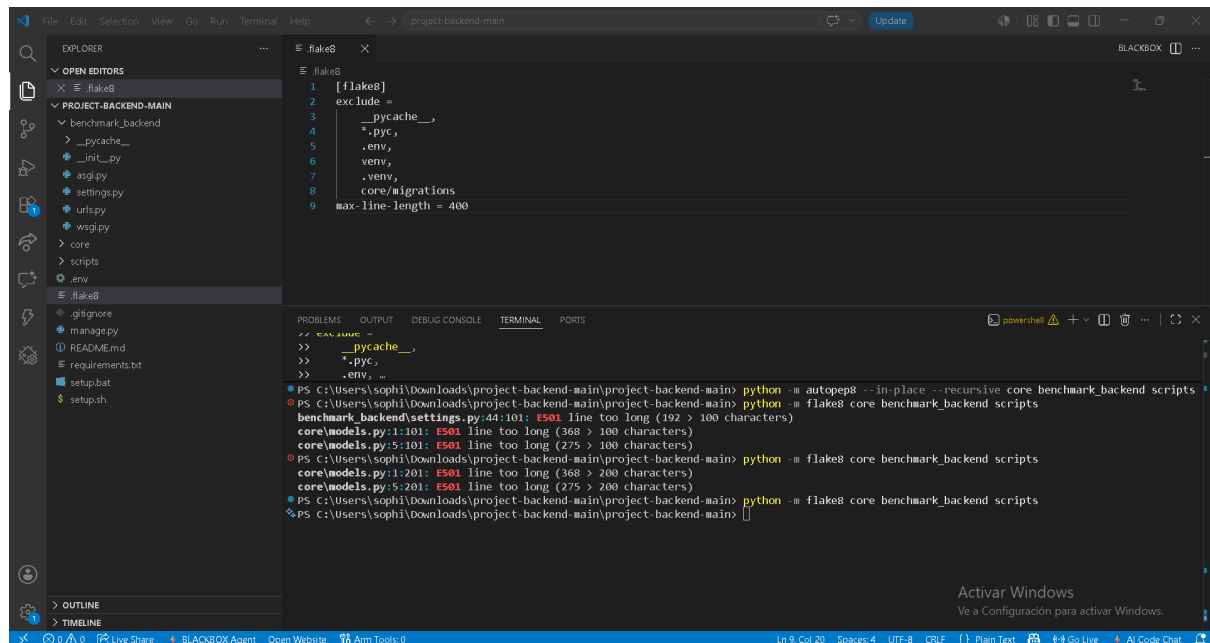
- Ejecución con archivo .flake8 mostrando reducción a 3 errores



```
File Edit Selection View Go Run Terminal Help
project-backend-main
EXPLORER
OPEN EDITORS
PROJECT-BACKEND-MAIN
  benchmark_backend
  core
  scripts
  .env
  .gitignore
  manage.py
  README.md
  requirements.txt
  setup.bat
  setup.sh
OUTLINE
TIMELINE
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\sophi\Downloads\project-backend-main\project-backend-main> python -m autopep8 --in-place --recursive core benchmark_backend scripts
PS C:\Users\sophi\Downloads\project-backend-main\project-backend-main> python -m flake8 core benchmark_backend scripts
benchmark_backend/settings.py:44:101: E501 line too long (192 > 100 characters)
core/models.py:1:101: E501 line too long (368 > 100 characters)
core/models.py:5:101: E501 line too long (275 > 100 characters)
PS C:\Users\sophi\Downloads\project-backend-main\project-backend-main>
.flake8
[flake8]
exclude =
  __pycache__,
  .pyc,
  .env,
  .venv,
  .venv,
  core/migrations
max-line-length = 100
```

5. Evidencia de corrección

Se adjunta pantallazo final donde, tras la aplicación de correcciones automáticas y ajustes en la configuración del linter, no se reportan errores en el código analizado.



6. Conclusión

El uso de herramientas de análisis estático permitió identificar y corregir problemas de estilo y calidad en el código del proyecto. La combinación de Flake8 para detección de errores y autopep8 para corrección automática facilitó la mejora del código, logrando finalmente un estado sin errores según las reglas definidas.